Assignment 1:

Analysis of SVD for Fine-Tuning LLMs

September 30, 2026

Big Data Analytics Techniques
and Applications

Fall 2026

Outline

- Kaggle Registration
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- Data Description
- Code Execution on Kaggle
- Report Content & Submission

Assignment 1

- **It is an individual work and counts for 5% of the semester grade**
- **Example Code Link:**

  https://github.com/nycubigdata-2026fall/BigData2026-Assignment1
- **Deadline:**
  - ~~October 14, 11:55 pm~~ → **October 18, 11:55 pm**
- **Task:**
  - Fine-tuning Llama-3.2-1B using SVD-based techniques to improve performance on mathematical question-answering tasks.
    - Use “Accuracy” as the evaluation metric.

Outline

- **Kaggle Registration**
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- Data Description
- Code Execution on Kaggle
- Report Content & Submission

Kaggle Registration

kaggle Competitions Benchmarks Game Arena Data Hub ••• Search Sign In **Register**

The World's AI Proving Ground

Discover what actually works in AI. Join 33M+ builders, researchers, and labs evaluating agents, models, and frontier technology through crowdsourced benchmarks, competitions, and hackathons.

Register with Google **Register with Email**

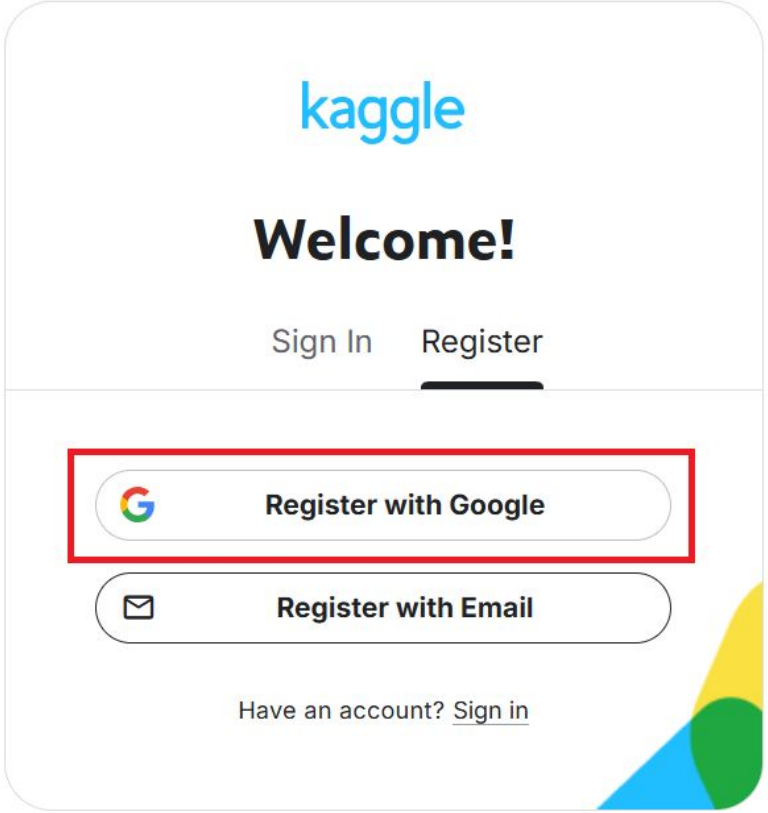


Kaggle Registration (cont.)

Since GPU time on Kaggle requires phone verification, if you:

- Already have a Kaggle account, you can just use that one
- If not, please register with your NYCU Email, since Assignment 3 would be a Kaggle competition that is limited to account with NYCU Emails

Please navigate to settings to finish the phone verification process



Outline

- Kaggle Registration
- **Hugging Face Registration**
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- Data Description
- Code Execution on Kaggle
- Report Content & Submission

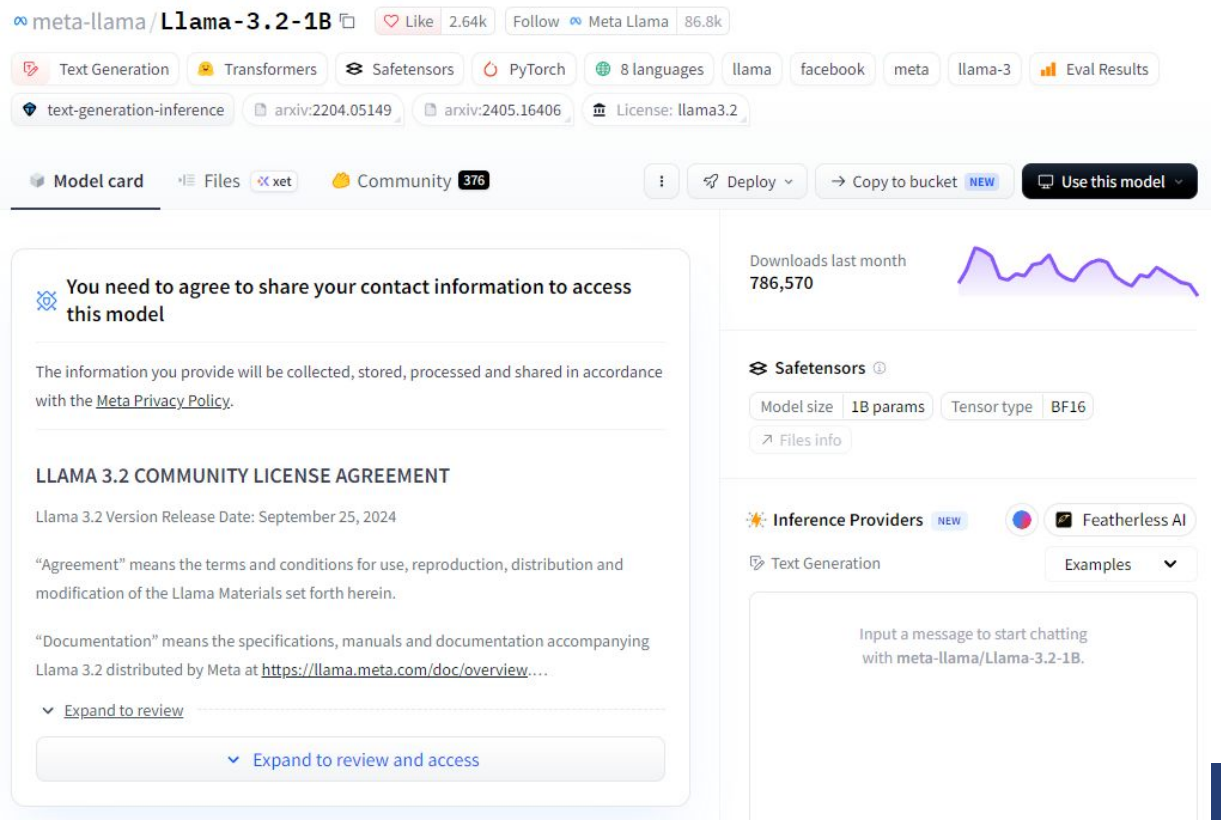
Hugging Face Registration

In Assignment 1, we use **Llama-3.2-1B** for fine-tuning:

- Llama-3.2-1B is a gated model
  - You will have to sign up a Hugging Face account and follow the following instructions.

Step 1.

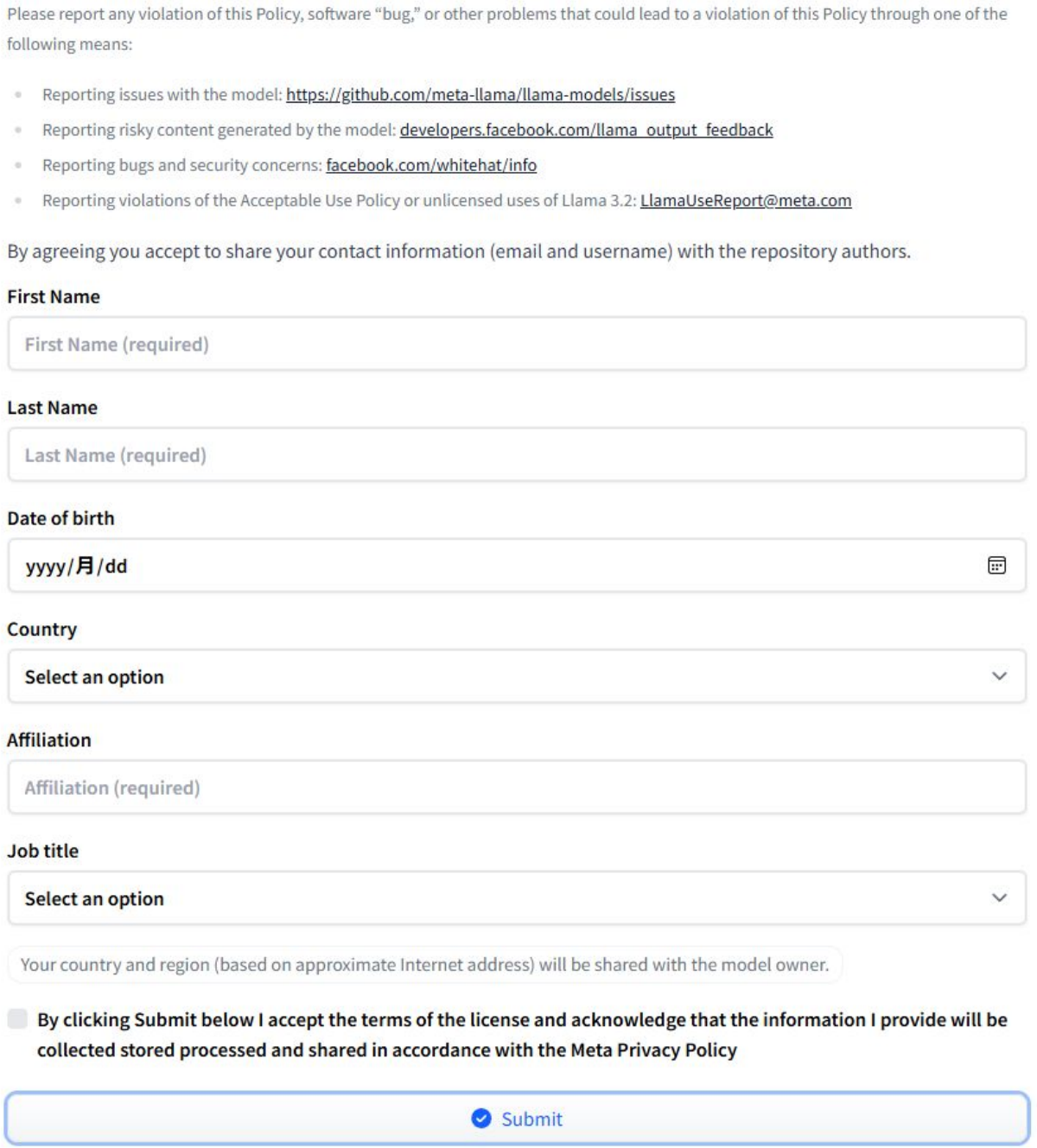
Search for Llama-3.2-1B on Hugging Face.



Hugging Face Registration (cont.)

Step 2.

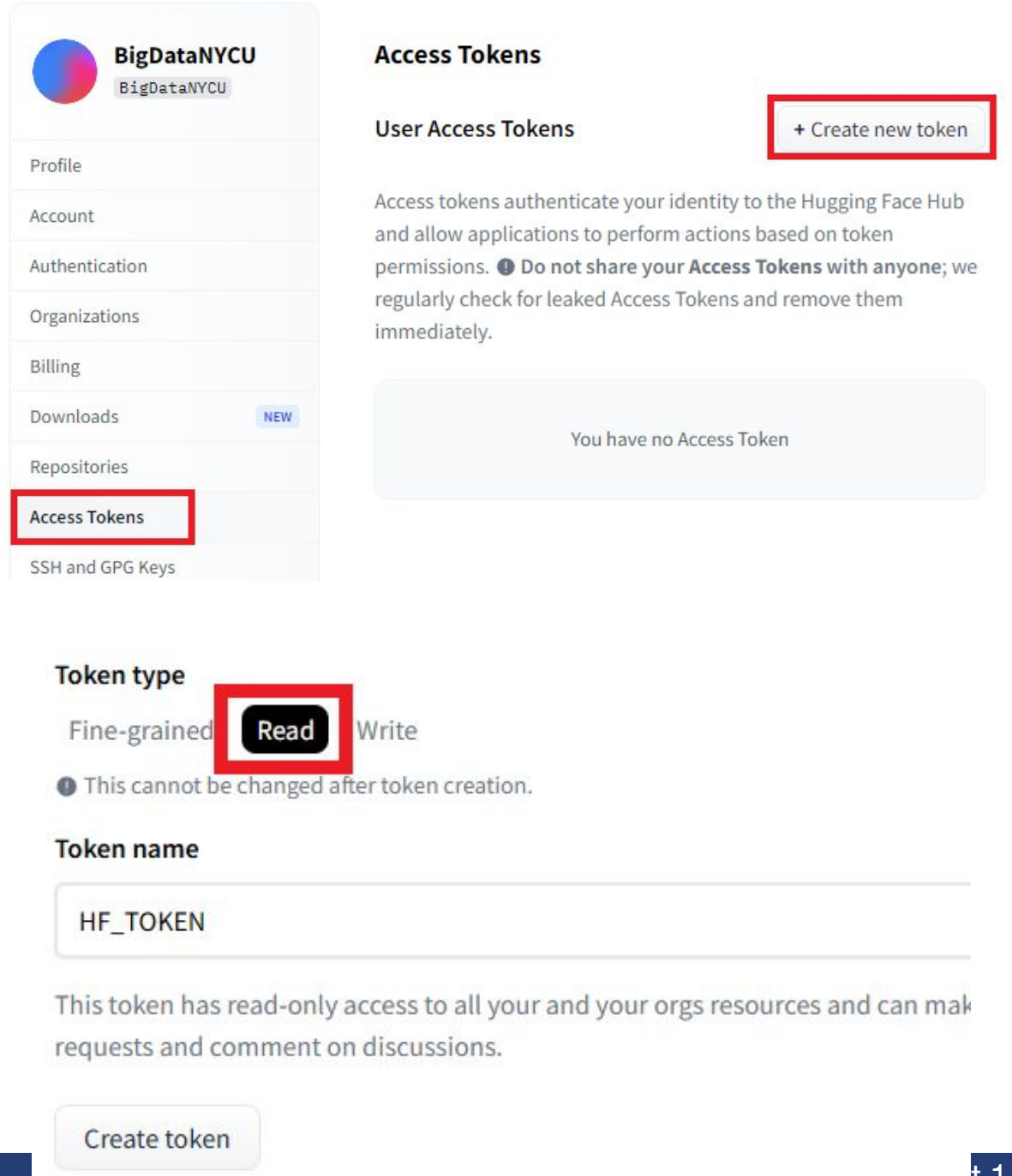
Fill in the form and submit.
It would take about half an hour to gain access to the model.



Hugging Face Registration (cont.)

Step 3.

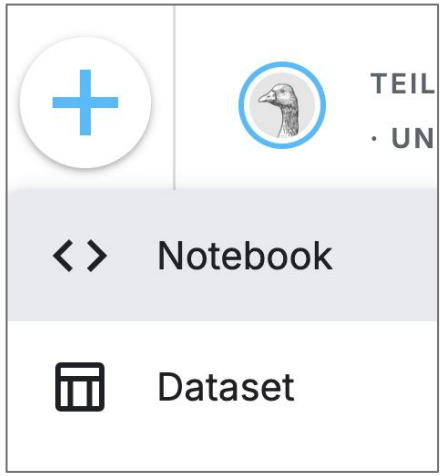
- Navigate to the settings and find Access Tokens
- Create a read type new token.
- Remember to keep your token well, you'll need to create a new one if lost.



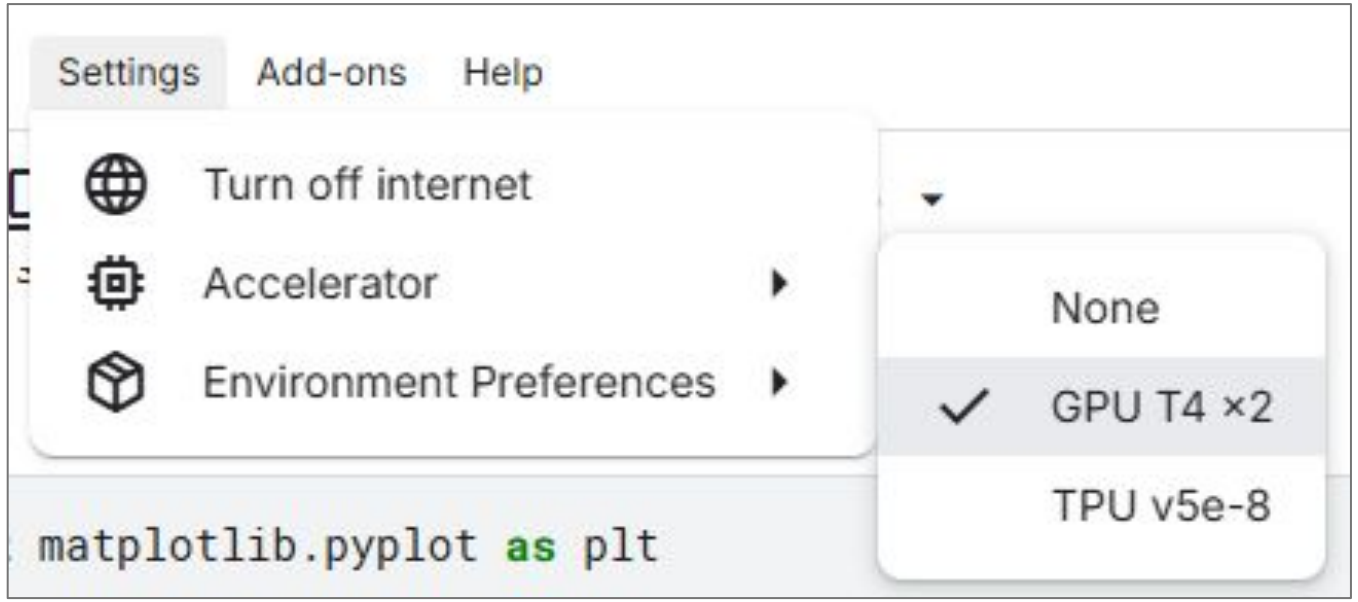
Outline

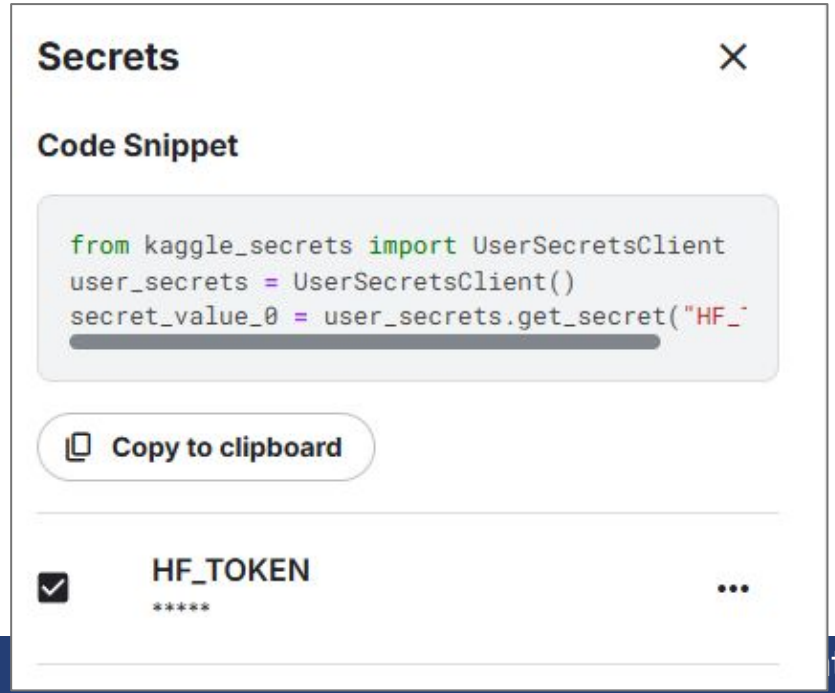
- Kaggle Registration
- Hugging Face Registration
- **Kaggle Notebook Setup**
- Code Description
- Basic Settings
- Data Description
- Code Execution on Kaggle
- Report Content & Submission

Kaggle Notebook Setup



1. Create a Notebook.
2. Click the button “Settings”, and set “Accelerator” to GPU T4 x2.
3. Click the button “Add-ons” and choose “Secrets”.
4. Click “Add secret”. Fill the name “**HF_TOKEN**” in the “LABEL” field, paste the token you created into “VALUE” field, and then click “Save”.





Code Description

BigData2026 Assignment 1 — Analysis of SVD for Fine-Tuning LLMs

This repository contains the implementation code modified based on the paper **PiSSA (Principal Singular Values and Singular Vectors Adaptation)** published at NeurIPS 2024 (For full paper information, please refer to "References").

The code is adapted from the original PiSSA implementation and modified to study how different parts of the singular-value spectrum affect low-rank fine-tuning.

Original work:

- Paper: PiSSA: Principal Singular Values and Singular Vectors Adaptation of Large Language Models
- Official code: MuLabPKU/PiSSA

File Description

- `finetune_q1-q3.ipynb` : File for Question 1 to Question 3 in Assignment 1.
- `finetune_q4.ipynb` : File for Question 4 in Assignment 1.

The two `.ipynb` files can be run independently.

Code Description

This code fine-tunes `meta-llama/Llama-3.2-1B` on `MetaMathQA` and evaluates it on two datasets: `GSM8K` and `MATH`.

Outline

- Kaggle Registration
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- **Basic Settings**
- Data Description
- Code Execution on Kaggle
- Report Content & Submission

Basic Settings in Assignment 1

Base model	Llama-3.2-1B
Target modules	“q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj”
Training Epochs	1
Learning rate	2e-05
Model precision	Train the model using FP16 precision.
Max output tokens	256
Parsing	The parsing function differs slightly from the original PiSSA repository and is specifically designed for Llama-3.2-1B, allowing for less strict output formatting.

This page shows the basic settings and the changes we made to the original repository.
You **do not need** to modify the settings above.

Outline

- Kaggle Registration
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- **Data Description**
- Code Execution on Kaggle
- Report Content & Submission

Dataset Description

- Datasets used in Assignment 1
  - **Fine-Tuning**
    - MetaMathQA [1]
  - **Evaluation**
    - GSM8K [2]
    - MATH [3]

[1] Yu, Longhui, et al. "Metamath: Bootstrap your own mathematical questions for large language models." *International Conference on Learning Representations*. Vol. 2024. 2024.
[2] Cobbe, Karl, et al. "Training verifiers to solve math word problems." *arXiv preprint arXiv:2110.14168* (2021).
[3] Hendrycks, Dan, et al. "Measuring mathematical problem solving with the math dataset." *arXiv preprint arXiv:2103.03874* (2021).

Dataset Description - MetaMathQA [Fine-Tuning]

- **Data Samples:**
  - 395,000 samples in total
  - For this assignment we use 25,000 samples

- **Description:**
  - MetaMathQA is used for **fine-tuning** the model in Assignment 1, where the data are augmented from the training sets of GSM8K and MATH.

- **Data Columns:**
  - type: Which dataset is this question from and what was changed.
  - query: The question used as input.
  - original_question: The question in the original dataset.
  - response: What the reply should be, where we parse the gold answer.

Dataset Description - MetaMathQA (Example)

A data sample of MetaMathQA:

```
type:  MATH_AnsAug

query:  Gracie and Joe are choosing numbers on the complex plane. Joe chooses the point $1+2i$. Gracie
chooses $-1+i$. How far apart are Gracie and Joe's points?

original_question:  Gracie and Joe are choosing numbers on the complex plane. Joe chooses the point $1
+2i$. Gracie chooses $-1+i$. How far apart are Gracie and Joe's points?

response:  The distance between two points $(x_1,y_1)$ and $(x_2,y_2)$ in the complex plane is given b
y the formula $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$.
In this case, Joe's point is $(1,2)$ and Gracie's point is $(-1,1)$.
So the distance between their points is $\sqrt{((-1)-(1))^2+((1)-(2))^2}=\sqrt{(-2)^2+(-1)^2}=\sqrt{4+
1}=\sqrt{5}$.
Therefore, Gracie and Joe's points are $\boxed{\sqrt{5}}$ units apart.
The answer is: \sqrt{5}
```

Dataset Description - GSM8K & MATH [Evaluation]

- **Data Samples:**
  - GSM8K and MATH contain 1,319 and 5,000 test samples, respectively.
  - For Assignment 1, we use 1,000 testing samples for each dataset.

- **Description:**
  - GSM8K and MATH are used for **evaluating** the model in Assignment 1
    - GSM8K contains simpler problems with mostly integer answers, while MATH includes more challenging problems with more complex answer formats.

- **Data Columns:**
  - instruction: Includes a instruction prompt and the question.
  - input: Left blank.
  - output: The answer for the question.
  - type: Whether it's from GSM8K or MATH.

Dataset Description - GSM8K (Example)

A data sample of GSM8K:

```
instruction:  Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
A robe takes 2 bolts of blue fiber and half that much white fiber.  How many bolts in total does it take?

### Response: Let's think step by step.

input:

output:  3

type:  gsm8k
```

Dataset Description - MATH (Example)

A data sample of MATH:

```
instruction:  Below is an instruction that describes a task. Write a response that appropriately completes the request.

### Instruction:
Find the matrix $\mathbf{M}$ such that
\[\mathbf{M} \begin{pmatrix} 1 & -2 \\ 1 & 4 \end{pmatrix} = \begin{pmatrix} 6 & 0 \\ 0 & 6 \end{pmatrix}.\]

### Response: Let's think step by step.

input:

output:  \begin{pmatrix} 4 & 2 \\ -1 & 1 \end{pmatrix}

type:  math
```

Outline

- Kaggle Registration
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- Data Description
- **Code Execution on Kaggle**
- Report Content & Submission

How to run the Provided Code on Kaggle

How to Run `finetune_q1-q3.ipynb` on Kaggle

1. Enable GPU

Open the notebook in Kaggle and enable a GPU accelerator.
Two T4 GPUs can be used through `DataParallel`, but the code also runs with one CUDA GPU.

2. Add Hugging Face Token

`Llama-3.2-1B` is a gated model.

1. Request access to `meta-llama/Llama-3.2-1B` on Hugging Face.
2. Create a Hugging Face access token.
3. In Kaggle, add the token as a secret named:

```
HF_TOKEN
```

How to Save the Output Result

Alternatively, you can fine-tune the original model again instead of reusing the model.

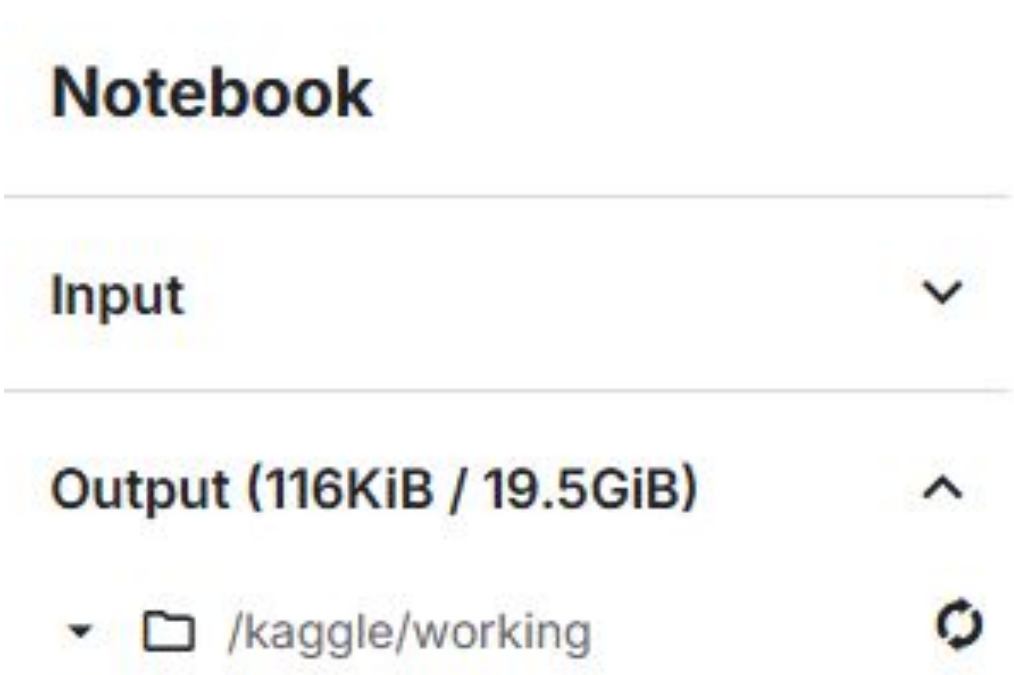
If you want to reuse the fine-tuned model, there are two ways:

- **If running directly:**
  - The model will be saved temporarily at `'/kaggle/working/Assignment1'`, where you can access via the sidebar.

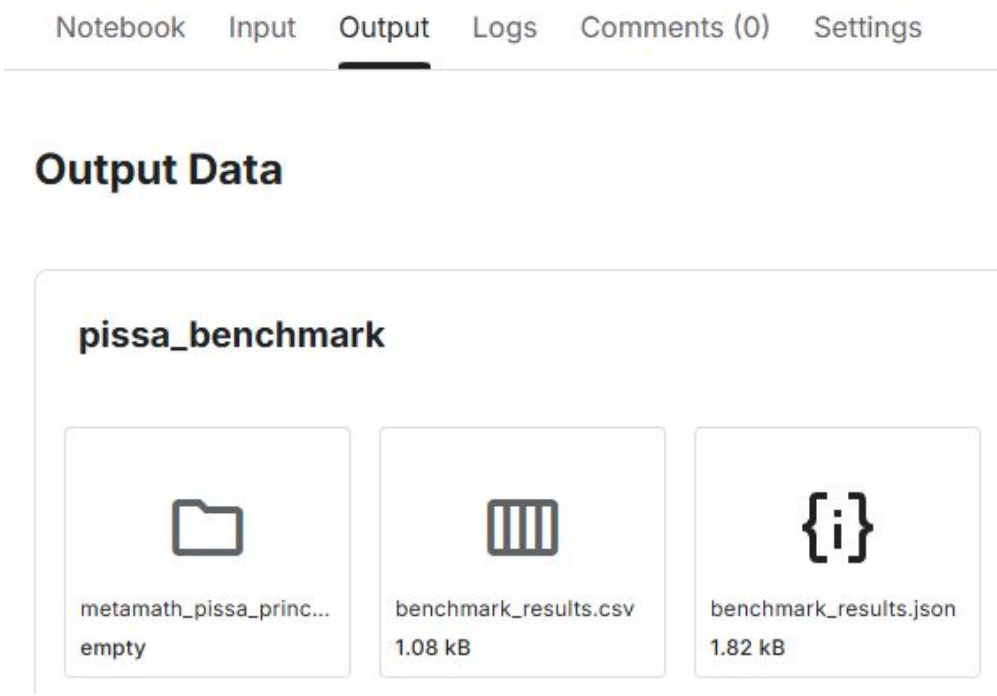
- **If using the button "Save Version"**:
  - The model would be saved in the folder "Output" with the same path

How to Save the Output Result (cont.)

- **If running directly:**



- **If using the button "Save Version":**



How to Save the Output Result (cont.)

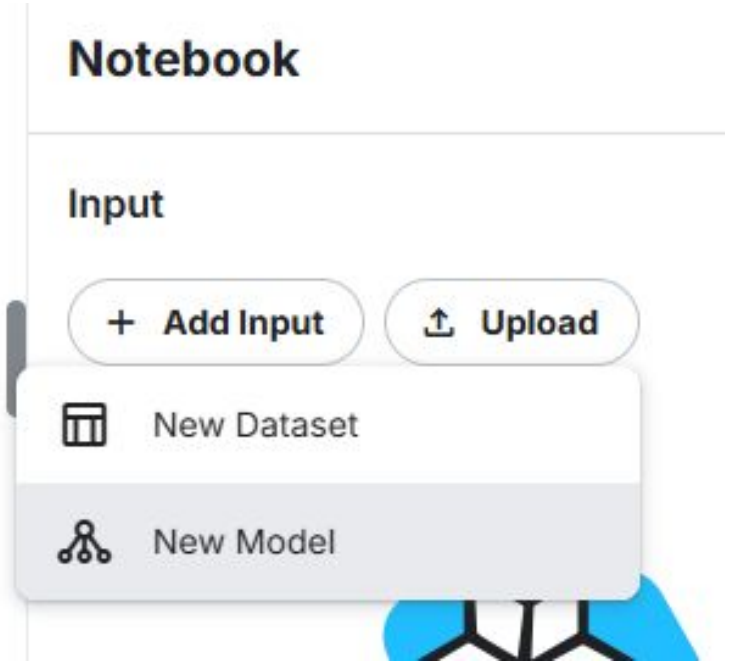
Important output files include:

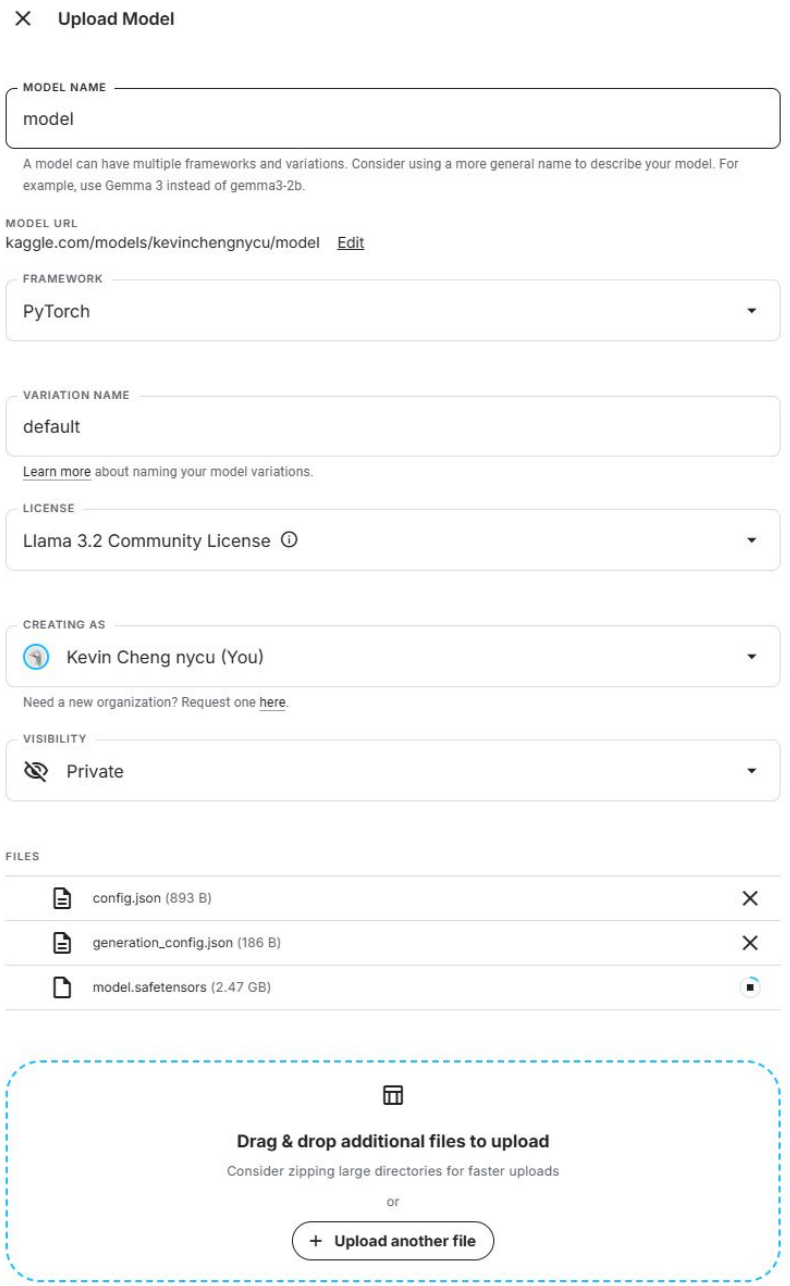
- Accuracy performance on two datasets:
  - File Name: `benchmark_results.csv`
  - Corresponding Column Names:
    - `gsm8k_accuracy`
    - `math_accuracy`

- Information of Fine-Tuned models:
  - Directory:
    `/metamath_pissa_{YOUR_COMPONENT_NAME}_r{YOUR_RANK}_full_model/`
  - File Names:
    - `config.json`
    - `generation_config.json`
    - `model.safetensor`

How to Upload the Trained Model

1. Via the sidebar, go to “Upload” → "New Model”
2. Upload the files and select PyTorch as framework
3. Select your model in “Add Input”
4. Use the model path as MODEL_ID





Outline

- Kaggle Registration
- Hugging Face Registration
- Kaggle Notebook Setup
- Code Description
- Basic Settings
- Data Description
- Code Execution on Kaggle
- **Report Content & Submission**

Grading Criteria

The grade for Assignment 1 is based on the **your report**.

- **Basic Requirements**

  (If **any of** the following steps missing, **the grade for Assignment 1 will be zero**.)

  - Please upload your **code** to public GitHub repository.
  - Please submit your **report** to E3 platform, including the GitHub link to your Assignment 1 repository.

- **Report Requirements** (**5%**)
  - Include the contents of each provided question, and answer each question point by point.
  - Page Limitation:
    - **No more than 3 pages**
  - File Format:
    - PDF (**A4** format, exported from Word)

Your report will be graded based on the **logic, correctness, and completeness** of your answers.

Report Content

Question 1

```
# TODO for Question 1
"""
`MANUAL_RUNS` is used to configure the SVD settings for fine-tuning the LLM.
   - 'rank': the target rank used for truncated SVD
   - 'component': the selected range of singular values, corresponding to the s
"""
MANUAL_RUNS = [
   {'rank': 16, 'component': 'default'},
]

"""
`RUN_FRESH_BASELINE` is used to evaluate the original LLM without fine-tuning.
   - True: runs the original LLM as the baseline.
   - False: skips the baseline evaluation.
"""
RUN_FRESH_BASELINE = True
```

- **(1%) Question 1:**

  a. Report the accuracy of **original** Llama-3.2-1B on both GSM8K and MATH datasets.
  - Hint: Keep `"RUN_FRESH_BASELINE"` set to `"True"`.

  b. **Fine-tune** Llama-3.2-1B by setting the target rank to **16, 64, and 256**, respectively, and report the accuracy on both GSM8K and MATH datasets.
  - Hint: Modify `"MANUAL_RUNS"` and keep `"component"` set to `"default"`.

  c. Provide your **observations and explanations** on the performance trends with different target ranks for both datasets.

*Example **format** to report the accuracy.*

Accuracy	Original LLM	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
GSM8K				
MATH				

Report Content
Question 2

```
# TODO for Question 2
"""
Hint: Following we have tagged the start and the end positions of training.
"""
```

- **(1%) Question 2:**

  a. Following Question 1, run experiments with target ranks of 16, 64, and 256, and record the **training time (minute)** for each setting.

  b. Provide your **observations and explanations** on the training-time trends with different target ranks.

*Example **format** to report the training time.*

	Fine-Tuned LLM (RANK = 16)	Fine-Tuned LLM (RANK = 64)	Fine-Tuned LLM (RANK = 256)
Training Time (min.)			

Report Content
Question 3

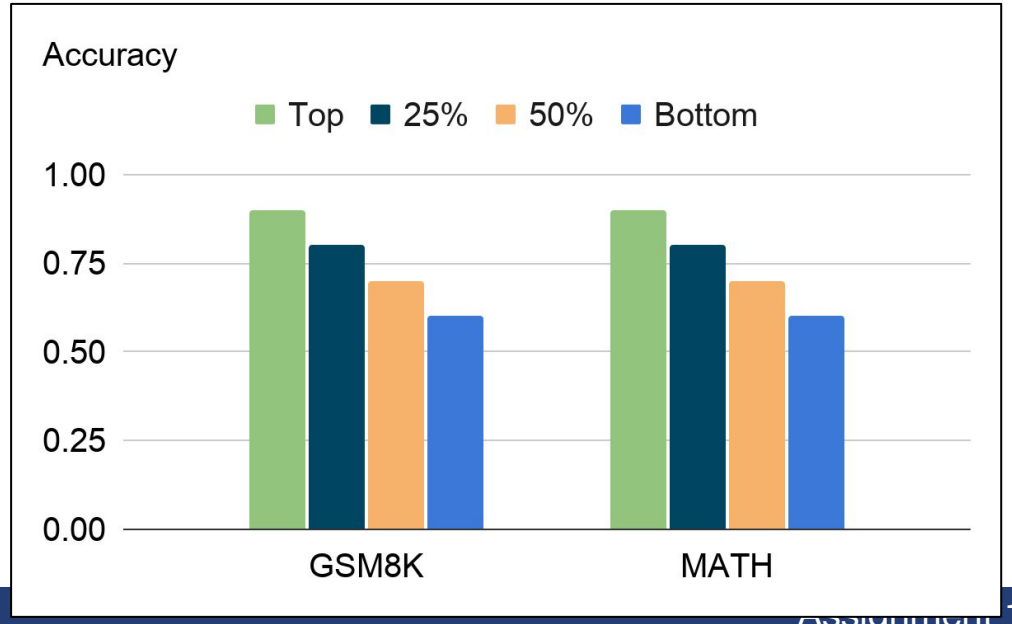
```
# TODO for Question 3
"""
Hint:
# Set the starting point at the 25% position of singular values S
 start = len(S) // 4

"""
```

- **(1%) Question 3:**

  a. For each dataset, perform SVD and fine-tune the model by selecting **four ranges of singular values** while fixing **Target Rank to 16**, and report the corresponding **accuracy in bar charts**.
  (1.) The **Top** 16 singular values (set `"component"` to `"default"`, which already exists in codes)
  (2.) 16 consecutive singular values starting at the **25%** position
  (3.) 16 consecutive singular values starting at the **50%** position
  (4.) The **Bottom** 16 singular values

  b. Provide your **observations and explanations** on the performance trends for both datasets.



*Example **bar chart** to report the accuracy.*

Report Content
Question 4

```
# TODO for Question 4
"""

Hint:
# Analyze the weights of the query projection layer (q_proj)
 q_weight = model.model.layers[k].self_attn.q_proj.weight.detach()

# Apply Full SVD to decompose the weight matrix into U (left singular vectors), S (singular va
 U, S, Vh = torch.linalg.svd(q_weight.float(), full_matrices=False)

# Compute the retained energy ratio (Cumulative energy) for different values of target ranks.
 energy_ratios = torch.cumsum(S ** 2, dim=0) / torch.sum(S ** 2) * 100
"""
```

- **(2%) Question 4:**

  No fine-tuning or evaluation is required. Analyze only the original Llama-3.2-1B model.

  a. Perform SVD on the **query projection (`"q_proj"`) weights** of the LLM layers **0, 7, 15**:

  i. No need to specify target rank

  b. For each layer, please report the **minimum target rank** required to retain **at least 50% cumulative energy.**

  c. Please provide the following **visualization for Layer 0, 7, 15, respectively:**
  (For clarity, visualize only the **first 200** singular values):

  i. Cumulative energy (%) using a **line chart**.

  ii. Singular value magnitudes using a **bar chart.**

  d. Provide your **observations** for the above results.

Report Content
Question 4 (cont.)

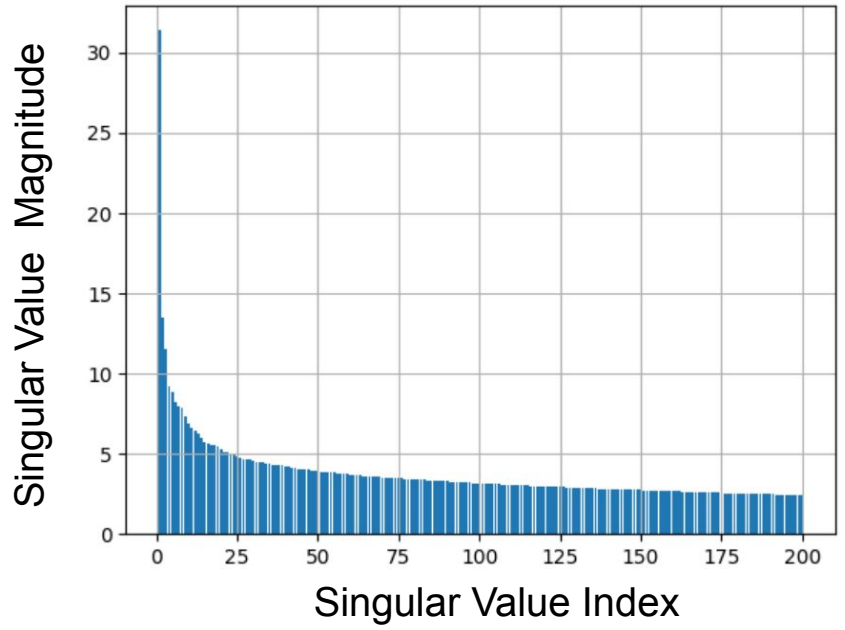
- **(2%) Question 4:**

You are required to provide a bar chart and a line chart for **each of** Layer 0, Layer 7, and Layer 15. (**6 charts in total**)

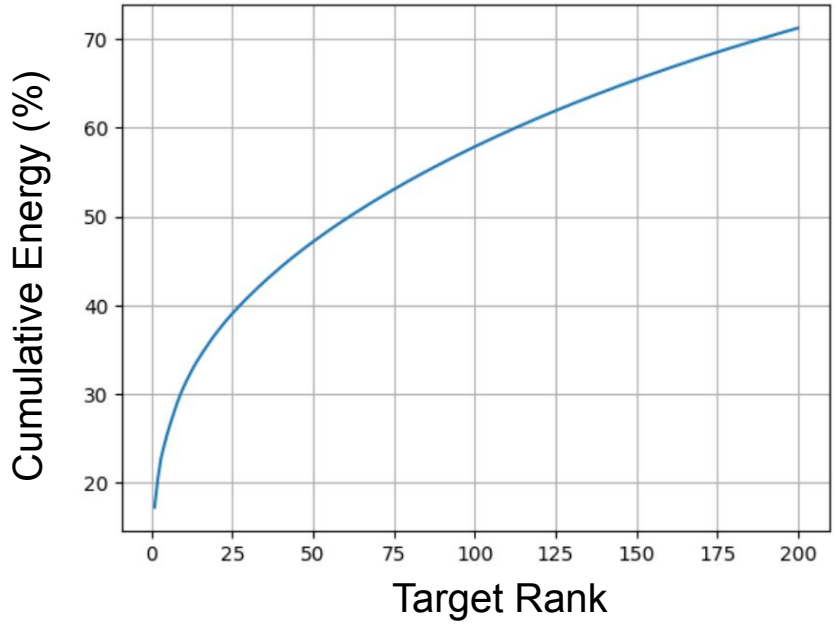
*Example **format** to report the minimum target rank.*

	Layer 0	Layer 7	Layer 15
Minimum Target Rank for query projection weights			

*Example **bar chart** for Singular Value Magnitude*



*Example **line chart** for Cumulative Energy*



Report and Code Submission

Please submit your report & code by **October 18, 11:55 PM.**

Submissions consist of two parts:

1. **Github Repo**
   a. Please host your **code on a public GitHub repository** (commit before deadline).
2. **Report.**
   a. Please ensure your public **GitHub link to Assignment 1 repository is included** within your report, and submit your **report via E3.**
   b. **File name:** Asg1_{studentID}.pdf (e.g., Asg1_314551051.pdf)

Please write down your report in **English.**

Report and Code Submission

- **Late submissions are not accepted.**
- You can work with AI tools to discuss and improve your own ideas, but:
  - Points will be **deducted** for **AI hallucinations** based on severity.
  - **Plagiarism** on assignments (either codes or report) will result in a **grade of 0.**

If you have any other questions, please feel free to contact the TA at nycu.bigdata@gmail.com

References

- Hu, Edward J., Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "Lora: Low-rank adaptation of large language models." *arXiv preprint arXiv:2106.09685* (2021).
  - Source Code: https://github.com/microsoft/LoRA

- Codes are modified from:
  - Meng, Fanxu, Zhaohui Wang, and Muhan Zhang. "Pissa: Principal singular values and singular vectors adaptation of large language models." *Advances in Neural Information Processing Systems* 37 (2024): 121038-121072.
  - Source Code: https://github.com/MuLabPKU/PiSSA